

C++ String Functions Cheat Sheet

This is a comprehensive reference of the most important and commonly used `std::string` functions in C++. Includes function names, parameter types, return types, and sample usage.

◆ Basic String Functions

Function	Parameters	Return Type	Description	Sample Usage
<code>length()</code> / <code>size()</code>	None	<code>size_t</code>	Returns the number of characters	<code>s.length()</code>
<code>empty()</code>	None	<code>bool</code>	Checks if string is empty	<code>s.empty()</code>
<code>clear()</code>	None	<code>void</code>	Removes all characters	<code>s.clear()</code>
<code>resize(n)</code>	<code>size_t n</code>	<code>void</code>	Changes size to <code>n</code>	<code>s.resize(5)</code>
<code>resize(n, ch)</code>	<code>size_t n</code> , <code>char ch</code>	<code>void</code>	Resizes and fills with <code>ch</code>	<code>s.resize(10, 'x')</code>
<code>capacity()</code>	None	<code>size_t</code>	Returns capacity	<code>s.capacity()</code>
<code>shrink_to_fit()</code>	None	<code>void</code>	Shrinks capacity	<code>s.shrink_to_fit()</code>

◆ Access Functions

Function	Parameters	Return Type	Description	Sample Usage
----------	------------	-------------	-------------	--------------

<code>at(pos)</code>	<code>size_t pos</code>	<code>char&</code>	Bounds-checked access	<code>s.at(2)</code>
<code>operator[]</code>	<code>size_t pos</code>	<code>char&</code>	Direct access (no bounds check)	<code>s[2]</code>
<code>front()</code>	None	<code>char&</code>	First character	<code>s.front()</code>
<code>back()</code>	None	<code>char&</code>	Last character	<code>s.back()</code>

◆ Modification Functions

Function	Parameters	Return Type	Description	Sample Usage
<code>append(str)</code>	<code>const string& str</code>	<code>string&</code>	Appends <code>str</code>	<code>s.append("abc")</code>
<code>push_back(ch)</code>	<code>char ch</code>	<code>void</code>	Adds <code>char</code> to end	<code>s.push_back('x')</code>
<code>pop_back()</code>	None	<code>void</code>	Removes last <code>char</code>	<code>s.pop_back()</code>
<code>insert(pos, str)</code>	<code>size_t pos, const string& str</code>	<code>string&</code>	Inserts at position	<code>s.insert(2, "xyz")</code>
<code>erase(pos, len)</code>	<code>size_t pos, size_t len</code>	<code>string&</code>	Erases part of string	<code>s.erase(1, 3)</code>
<code>replace(pos, len, str)</code>	<code>size_t pos, size_t len, const string& str</code>	<code>string&</code>	Replace portion with <code>str</code>	<code>s.replace(0, 2, "hi")</code>
<code>swap(str2)</code>	<code>string& str2</code>	<code>void</code>	Swaps with <code>str2</code>	<code>s.swap(s2)</code>

▲ Searching Functions

▼ Searching Functions

Function	Parameters	Return Type	Description	Sample Usage
<code>find(str)</code>	<code>const string& str</code>	<code>size_t</code>	First occurrence	<code>s.find("abc")</code>
<code>find(ch)</code>	<code>char ch</code>	<code>size_t</code>	First occurrence of <code>ch</code>	<code>s.find('a')</code>
<code>rfind(str)</code>	<code>const string& str</code>	<code>size_t</code>	Last occurrence	<code>s.rfind("abc")</code>
<code>find_first_of(str)</code>	<code>const string& str</code>	<code>size_t</code>	Any of characters	<code>s.find_first_of("aei")</code>
<code>find_last_of(str)</code>	<code>const string& str</code>	<code>size_t</code>	Last of characters	<code>s.find_last_of("aei")</code>
<code>find_first_not_of(str)</code>	<code>const string& str</code>	<code>size_t</code>	First not in <code>str</code>	<code>s.find_first_not_of("aei")</code>
<code>find_last_not_of(str)</code>	<code>const string& str</code>	<code>size_t</code>	Last not in <code>str</code>	<code>s.find_last_not_of("aei")</code>

✦ Returns `string::npos` if not found.

◆ Substring Functions

Function	Parameters	Return Type	Description	Sample Usage
			From <code>pos</code> to	

<code>substr(pos)</code>	<code>size_t pos</code>	<code>string</code>	From <code>pos</code> for <code>len</code>	<code>s.substr(2)</code>
<code>substr(pos, len)</code>	<code>size_t pos, size_t len</code>	<code>string</code>	From <code>pos</code> for <code>len</code>	<code>s.substr(1, 3)</code>

◆ Comparison Functions

Function	Parameters	Return Type	Description	Sample Usage
<code>compare(str)</code>	<code>const string& str</code>	<code>int</code>	Lexicographical compare	<code>s.compare("abc")</code>
<code>==, !=, <, <=, >, >=</code>	<code>string</code>	<code>bool</code>	Operators for comparison	<code>if (s1 < s2)</code>

◆ Conversion Functions

Function	Parameters	Return Type	Description	Sample Usage
<code>c_str()</code>	None	<code>const char*</code>	C-style string	<code>puts(s.c_str())</code>
<code>data()</code>	None	<code>const char*</code>	Same as <code>c_str()</code>	<code>s.data()</code>
<code>stoi(s)</code>	<code>string</code>	<code>int</code>	String to int	<code>stoi("123")</code>
<code>to_string(n)</code>	<code>number</code>	<code>string</code>	Int/float to string	<code>to_string(42)</code>

◆ Iterators

Function	Return Type	Description
<code>begin()</code>	<code>iterator</code>	Start iterator

<code>begin()</code>	<code>iterator</code>	Start iterator
<code>end()</code>	<code>iterator</code>	End iterator
<code>rbegin()</code>	<code>reverse_iterator</code>	Reverse start
<code>rend()</code>	<code>reverse_iterator</code>	Reverse end

◆ STL Algorithms (Useful with Strings)

Function	Description	Sample Usage
<code>reverse(s.begin(), s.end())</code>	Reverses string	<code>reverse(s.begin(), s.end())</code>
<code>sort(s.begin(), s.end())</code>	Sorts string	<code>sort(s.begin(), s.end())</code>
<code>count(s.begin(), s.end(), ch)</code>	Count occurrences of <code>ch</code>	<code>count(s.begin(), s.end(), 'a')</code>

✓ Sample Code

```
#include <iostream>
#include <string>
#include <algorithm>
using namespace std;

int main() {
    string s = "hello";

    s.push_back('x');          // hellox
    s.append(" world");        // hellox world
    s.replace(0, 5, "Hi");      // Hix world
    string sub = s.substr(4, 3); // wor

    if (s.find("world") != string::npos)
        cout << "Found 'world'" << endl;

    reverse(s.begin(), s.end()); // dlrow x iH
    cout << s << endl;
}
```

```
    return 0;  
}
```